Un moteur d'inférence pour eForth

version 1.0 - 15. octobre 2026



Marc PETREMANN

Table des matières

Introduction **3**

L'environnement de développement **4**

Le fichier INFERENCE.FS 4

Le projet main.fs 4

Les autres fichiers 5

Définir des faits **6**

Créer un fait 6

Introduction

Un **moteur d'inférence** est le composant central d'un système expert ou d'une intelligence artificielle symbolique, responsable de la prise de décision et du raisonnement automatisé. Il fonctionne en combinant une **base de connaissances** (constituée de faits et de règles de production) avec des algorithmes de déduction pour en déduire de nouvelles informations.

Il utilise principalement deux stratégies de raisonnement :

- **Le chaînage avant**, qui part des faits disponibles pour arriver à une conclusion.
- **Le chaînage arrière**, qui part d'une hypothèse cible pour remonter aux faits nécessaires qui la valident.

En somme, il simule la logique humaine pour résoudre des problèmes complexes sans nécessiter de programmation procédurale rigide.

L’environnement de développement

Pour le moteur d’inférence, nous utiliserons la version eForth Windows. C’est une version FORTH utilisable pour Windows, disponible ici :

https://github.com/MPETREMANN11/eForth-Windows/tree/main/__program

La documentation de cette version est accessible ici :

https://github.com/MPETREMANN11/eForth-Windows/tree/main/__documentation

eForth ne nécessite aucune installation.

Téléchargez simplement le fichier **uEf64-7.0.7.21.exe** dans un répertoire, à l’emplacement de votre choix, nommé par exemple **eForth**.

Ouvrez ce répertoire **eForth**. Créez un nouveau dossier **INFERENCE**. Téléchargez les fichiers nécessaires depuis le dépôt logiciel. Vous devriez avoir :

```
📁 eForth
    uEf64-7.0.7.21.exe
    INFERENCE.fs
    📁 INFERENCE
        config.fs
        facts.fs
        inference-engine.fs
        main.fs
        rules.fs
        tests.fs
```

Le fichier INFERENCE.FS

C’est le fichier qui permet de se connecter au reste du projet. Contenu :

```
\ load INFERENCE
s" INFERENCE/main.fs" included
```

A chaque fois que vous voudrez compiler le code et utiliser le moteur d’inférence, il suffira de lancer **eForth** en cliquant sur le fichier **uEf64-7.0.7.21.exe**. Une fois eForth démarré, tapez :

```
include INFERENCE.fs
```

Le projet main.fs

L’ensemble du projet est compilé depuis **main.fs**. Exemple de contenu :

```
include config.fs               \ load config
include inference-engine.fs     \ load inference engine
```

Vous pouvez rajouter le chargement d’autres fichiers. Par exemple, pour compilet **tests.fs** :

```
include config.fs               \ load config
include inference-engine.fs     \ load inference engine
include tests.fs
```

Les autres fichiers

Ils sont chargés depuis **main.fs**. Le projet est découpé en fonctionnalités utilisables selon l'évolution du projet. La liste et le contenu des fichiers sources peut évoluer sans préavis et ne plus être ceux décrits dans ce document.

L'ordre de chargement des fichiers est essentiel. Ainsi, on ne peut pas charger le contenu de **facts.fs** avant d'avoir traité le contenu de **inference-engine.fs**.

Définir des faits

Vous pensez maîtriser la logique ? C'est parfait. Alors vous savez manipuler les opérateurs logiques `ET`, `OU`, `NON`… Alors voyons un cas pratique avec un exemple tout à fait trivial :

- la température est très froide
- il pleut

Nous avons deux faits. On peut déduire que s'il pleut et qu'il fait très froid, le risque de gel est réel.

On peut coder ceci en FORTH :

```
 0 constant FALSE
-1 constant TRUE

VARIABLE fait-pluie
VARIABLE fait-froid
VARIABLE fait-gel

: RAZ-FAITS ( -- )
    FALSE fait-pluie !
    FALSE fait-froid !
    FALSE  fait-gel ! ;
```

Nos deux variables `fait-pluie` et `fait-gel` sont les drapeaux à tester, avec les valeurs 0 pour faux, -1 pour vrai. Sur ces deux faits, nous pouvons appliquer ces règles :

```
\ Définition des règles comme des mots Forth standards
: cond-regle1 ( -- flag )
    fait-pluie @  fait-froid @  AND ;
: act-regle1  ( -- )
    cond-regle1 fait-gel ! ;
```

A ce stade, nous ne pouvons pas encore parler d'inférence. Et une seule règle, ici `act-regle1` ne justifie pas la création d'un moteur d'inférence.

Créer un fait

C'est le premier pas vers un moteur d'inférence. Un fait est une variable qui contient un état logique ou une valeur numérique. Pour définir un fait :

```
: FACT: ( n|fl comp: -- <word> | exec: -- n )
    value
  ;
```

Ce qui nous permet de réécrire les variables ainsi :

```
FALSE FACT: fait-pluie
FALSE FACT: fait-froid
FALSE FACT: fait-gel
```